

Cybersource B2C Commerce - SOAP Authentication and Encryption Guide

Contents

Introduction.....	2
1. Merchants using ENT cartridge v21.1.0 and above	3
1.1. Steps for managing P12 certificate	3
Step 1: Create P12 file	3
Option 1: Create a p12 key.....	3
Option 2: Create Meta Key.	3
We can assign a single meta key to dozens or hundreds of transacting MIDs simultaneously.....	3
Step 2: Place the Keystore file in Business Manager for SOAP Authentication	4
Step 3: Import .crt to Business Manager for MLE	4
Step 4: Configurations in Business Manager.....	4
MLE	5
1.2. Code changes for MLE.....	6
Step 1: Changes to read the newly added configurations from Business Manager	6
Step 2: Changes in SoapServiceInit.js file.....	7
Step 3: Metadata changes to create configurations	8
2. Merchants using Site Genesis cartridge v21.1.0 and above	10
3. Merchants using cartridge version older than v21.1.0	10

Introduction

This document provides a step-by-step guide for managing P12 certificates and code changes for MLE implementation without upgrading the cartridge. Also includes the changes in configuration to make it general for both Authentication and MLE. Additionally, it outlines the essential code modifications required to ensure seamless integration and functionality.

Message-Level Encryption (MLE) enables you to store information or communicate with other parties while helping to prevent uninvolved parties from understanding the stored information. MLE is optional and supported only for payments services.

Meta key is a specialized API key that a portfolio or merchant account user can create for the purposes of processing transactions on behalf of multiple of their transacting MID accounts. Meta keys are useful for organizations whose transacting MID users do not manage or store their own individual API keys. Instead of having to create and assign a unique API key for each of your transacting MIDs, you can create and assign a single meta key to dozens or hundreds of your transacting MIDs simultaneously.

1. Merchants using ENT cartridge v21.1.0 and above

1.1.Steps for managing P12 certificate

Below are the steps to generate a P12 file or meta key and managing it.

Step 1: Create P12 file

Option 1: Create a p12 key.

We can use p12 key generated for a specific MID from which it is created.

1. Follow steps mentioned in the [link](#) to generate a P12 certificate in Business Center.
2. Make a note of password set to the P12 key.
3. Download the generated P12 file.

Option 2: Create Meta Key.

We can assign a single meta key to dozens or hundreds of transacting MIDs simultaneously.

1. Follow steps mentioned in the [link](#) to generate a meta key from Business Center.
2. Make a note of password set to the meta key.
3. Download the generated p12 file.

Step 2: Place the Keystore file in Business Manager for SOAP Authentication

- Login to Business Manager and Navigate to Administration > Operations > Private Keys and Certificates.
- Click on Import and select the p12 file generated in [Step1](#).
- Add Alias to the p12 key. Make note of the alias and use it for custom preference CsAuth_Alias

- Enter Source password used while generating the p12.
- Click on Save.

Step 3: Import .crt to Business Manager for MLE

- Run below openssl command to extract the certificate from p12 file generated in [Step1](#).
openssl pkcs12 -in yourfile.p12 -cacerts -nokeys -out certificate.crt
- Open this file in a text editor and make sure the `CyberSource_SJC_US` is the first certificate.
- Login to Business Manager and Navigate to Administration > Operations > Private Keys and Certificates.
- Click on Import and select the .crt file generated in above step.
- Add an Alias to the certificate imported. Make note of the alias and use it for custom preference CsMLE_Alias
- Click on Save

Note: If we try importing multiple certificates with same content of different CyberSource MID's then Salesforce throws an error stating that the certificate already exists with a different alias. We can use the same certificate for multiple mid's

Step 4: Configurations in Business Manager

Refer section [Metadata changes to create configurations](#) to create required configurations.

Go to **Merchant Tools > Site Preferences > Custom Preferences > Cybersource: Core** and set values for the following parameters

Field	Description
Cybersource Merchant ID	Cybersource Merchant ID (mention the merchant ID not the account or portfolio id)
CsAuth_Alias	Alias specified for p12 file imported in Business Manager in Step2 for Authentication.

Go to **Merchant Tools > Site Preferences > Custom Preferences > CyberSource: Message Level Encryption** and set values for the following parameters

Field	Description
CsMLE_Alias	Alias of the certificate imported to Business Manager in Step3 .

3

Refer to the below example.

Allas of the private key for Authentication

wlproltd_p12

Alias of the private key in Private Keys and Certificates for Authentication

CyberSource: Message Level Encryption ?

Cancel

[Apply to Other Sites](#)

Save

Search by ID

Q

Y

1-2 ▾ of 2

Name	Value	Default Value
Alias of the certificate in Private Keys and Certificate for Request MLE		
(CsMLE_Alias) (String)		cybersource_sjc_us
Alias of the certificate in Private Keys and Certificate for Request MLE	Alias of the certificate in Private Keys and Certificate for Request MLE	

Administration / Operations /

Private Keys and Certificates ?

Search for alias or hostname... 

Delete

☐ Alias	Hostnames	Type	Valid From	Valid To	Import Date
☐ soon_expired.crt		Trusted Certificate	9/17/2025	9/18/2025	9/17/2025
☐ soon_expired.p12		Private Key	9/17/2025	9/18/2025	9/17/2025
☐ wiproltd.crt		Trusted Certificate	7/26/2023	7/26/2026	9/18/2025
☐ wiproltd.p12		Private Key	11/21/2024	11/21/2026	9/18/2025

1.2.Code changes for MLE

Step 1: Changes to read the newly added configurations from Business Manager

Make the following changes to **libCybersource.js** file.

Path: “cartridges/int_cybersource_sfra/cartridge/scripts/cybersource/libCybersource.js”

1. Add the following functions to read the new configurations in “CybersourceHelper” object.

```
getAliasForMLE: function () {
    return Site.getCurrent().getCustomPreferenceValue('CsMLE_Alias');
},
```

2. Add the following changes to existing configurations for Authentication.

```
getAliasForSignature: function () {
    return Site.getCurrent().getCustomPreferenceValue('CsAuth_Alias');
},
```

Step 2: Changes in SoapServiceInit.js file

Path: “cartridges/int_cybersource_sfra/cartridge/scripts/init/SoapServiceInit.js”

Replace the existing **execute()** method with the following method in “CyberSourceTransactionService” service.

```
execute: function (svc, parameter) {
    var WSU_NS = "http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-
wssecurity-utility-1.0.xsd";
    var libCybersource =
require('*/cartridge/scripts/cybersource/libCybersource');
    var CybersourceHelper = libCybersource.getCybersourceHelper();
    var alisForSignature = CybersourceHelper.getAliasForSignature();
```

```

var aliasForEncryption = CybersourceHelper.getAliasForMLE();

var secretsMap = new HashMap();

//request-config
var requestCfg = new HashMap();

    requestCfg.put(WSUtil.WS_ACTION, WSUtil.WS_TIMESTAMP + " " +
WSUtil.WS_SIGNATURE + " " + WSUtil.WS_ENCRYPT);
    // define encryption properties
    requestCfg.put(WSUtil.WS_ENCRYPTION_USER, aliasForEncryption);
    requestCfg.put(WSUtil.WS_ENC_PROP_KEYSTORE_TYPE, "managed");
    requestCfg.put(WSUtil.WS_ENC_PROP_KEYSTORE_ALIAS,
aliasForEncryption);
    requestCfg.put(WSUtil.WS_ENC_KEY_ID,
WSUtil.KEY_ID_TYPE_X509_KEY_IDENTIFIER);

    requestCfg.put(
        WSUtil.WS_ENCRYPTION_PARTS,
        "{Element}{" +
        WSU_NS +
        "}Timestamp;" +
        "{Content}{http://schemas.xmlsoap.org/soap/envelope/}Body",
    );
}
else {
    requestCfg.put(WSUtil.WS_ACTION, WSUtil.WS_TIMESTAMP + " " +
WSUtil.WS_SIGNATURE);
}
requestCfg.put(WSUtil.WS_SIGNATURE_USER, alisForSignature);
requestCfg.put(WSUtil.WS_PASSWORD_TYPE, WSUtil.WS_PW_TEXT);
requestCfg.put(WSUtil.WS_SIG_DIGEST_ALGO,
"http://www.w3.org/2001/04/xmlenc#sha256");

// define signature properties
requestCfg.put(WSUtil.WS_SIG_PROP_KEYSTORE_TYPE, "managed");
requestCfg.put(WSUtil.WS_SIG_PROP_KEYSTORE_ALIAS, alisForSignature);
requestCfg.put(WSUtil.WS_SIGNATURE_PARTS,
"{Element}{http://schemas.xmlsoap.org/soap/envelope/}Body");
requestCfg.put(WSUtil.WS_SIG_KEY_ID,
WSUtil.KEY_ID_TYPE_DIRECT_REFERENCE);

requestCfg.put(WSUtil.WS_SECRETS_MAP, secretsMap);

```



```

//response-config
var responseCfg = new HashMap();
responseCfg.put(WSUtil.WS_ACTION, WSUtil.WS_TIMESTAMP);

WSUtil.setWSSecurityConfig(svc.serviceClient, requestCfg, responseCfg);
// Setting WS security
return svc.serviceClient.runTransaction(parameter.request);
},

```

Step 3: Metadata changes to create configurations

1. Create file in “**metadata/sfra_meta/meta/Cybersource_MLE.xml**” file:

Add these following to the xml.

```

<?xml version="1.0" encoding="UTF-8"?>
<metadata xmlns="http://www.demandware.com/xml/impex/metadata/2006-10-31">
  <type-extension type-id="SitePreferences">
    <custom-attribute-definitions>
      <attribute-definition attribute-id="CsMLE_Alias">
        <display-name xml:lang="x-default">Alias of the certificate in Private Keys and Certificate for Request
MLE</display-name>
        <description xml:lang="x-default">Alias of the certificate in Private Keys and Certificate for Request
MLE</description>
        <type>string</type>
        <mandatory-flag>false</mandatory-flag>
        <externally-managed-flag>false</externally-managed-flag>
        <min-length>0</min-length>
        <default-value>cybersource_sjc_us</default-value>
      </attribute-definition>
    </custom-attribute-definitions>
    <group-definitions>
      <attribute-group group-id="Cybersource_MLE">
        <display-name xml:lang="x-default">CyberSource: Message Level Encryption</display-name>
        <attribute attribute-id="CsMLE_Alias"/>
      </attribute-group>
    </group-definitions>
  </type-extension>
</metadata>

```

2. If you do not have the old configurations (v24.1.3), you can directly add the new ones with the code below in file Make changes to “**metadata/sfra_meta/meta/Cybersource.xml**” file:

Add these new configurations code in **<custom-attribute-definitions>** element of the xml.

```
<attribute-definition attribute-id="CsAuth_Alias">
  <display-name xml:lang="x-default"> >Alias of the certificate for
Authentication </display-name>
  <description xml:lang="x-default"> >Alias of the certificate for
Authentication </description>
  <type>string</type>
  <mandatory-flag>false</mandatory-flag>
  <externally-managed-flag>false</externally-managed-flag>
  <min-length>0</min-length>
</attribute-definition>
```

Replace the following **<group-definitions>** with the existing one.

```
<group-definitions>
  <attribute-group group-id="CyberSource">
    <display-name xml:lang="x-default">CyberSource: Core</display-name>
    <attribute attribute-id="IsCartridgeEnabled" />
    <attribute attribute-id="CsMerchantId" />
    <attribute attribute-id="CsEndpoint" />
    <attribute attribute-id="CsDeveloperID" />
    <attribute attribute-id="CsDebugCybersource" />
    <attribute attribute-id="csMasterCardAuthIndicator" />
    <attribute attribute-id="CsAuth_Alias" />
  </attribute-group>
</group-definitions>
```

1. Add the below code in "metadata/sfra_meta/sites/Refarch/preferences.xml" file

```
<preference preference-id="CsMLE_Alias"></preference>
<preference preference-id="CsAuth_Alias">merchantid</preference>
```

2. Merchants using Site Genesis cartridge v21.1.0 and above

Merchants using Site Genesis cartridge v21.1.0 and above can follow the steps mentioned in [section 1](#).

NOTE: When referring to any file, use the path 'cartridges/int_cybersource/' instead of 'cartridges/int_cybersource_sfra/'."s

3. Merchants using cartridge version older than v21.1.0

We strongly recommend merchants using older versions of our cartridge to upgrade to our latest cartridge version as the older version contains deprecated packages and methods which may not be compatible with our latest changes.

However, please follow the steps below to update required files to use p12 authentications and MLE.

Step 1: Update folder name from webreference to webreferences2.

Change all the references of webreference to webreferences2 in our cartridge.

Step 2: Add below changes to SoapServiceInit.js

```

@@ -6,7 +6,7 @@
6      6      /****/
7      7      var dwsvc = require ("dw/svc");
8      8      var HashMap = require ("dw/util/HashMap");
9      9      - var SOAPUtil = require ("dw/rpc/SOAPUtil");
          9      + var WSUtil = require ('dw/ws/WSUtil');
10     10      var LocalServiceRegistry = require('dw/svc/LocalServiceRegistry');
11     11      /**

```

The SOAPUtil class is deprecated. Replace it with WSUtil. Replace the execute() function of “CyberSourceTransactionService” with below code snippet.

```

execute: function (svc, parameter) {

    var WSU_NS = "http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-
wssecurity-utility-1.0.xsd";

    var libCybersource =
require('*/cartridge/scripts/cybersource/libCybersource');

    var CybersourceHelper = libCybersource.getCybersourceHelper();

    var alisForSignature = CybersourceHelper.getAliasForSignature();

    var aliasForEncryption = CybersourceHelper.getAliasForMLE();

    var secretsMap = new HashMap();

    //request-config

    var requestCfg = new HashMap();

    requestCfg.put(WSUtil.WS_ACTION, WSUtil.WS_TIMESTAMP + " " +
WSUtil.WS_SIGNATURE + " " + WSUtil.WS_ENCRYPT);

    // define encryption properties

```

```

        requestCfg.put(WSUtil.WS_ENCRYPTION_USER, aliasForEncryption);

        requestCfg.put(WSUtil.WS_ENC_PROP_KEYSTORE_TYPE, "managed");

        requestCfg.put(WSUtil.WS_ENC_PROP_KEYSTORE_ALIAS,
aliasForEncryption);

        requestCfg.put(WSUtil.WS_ENC_KEY_ID,
WSUtil.KEY_ID_TYPE_X509_KEY_IDENTIFIER);


        requestCfg.put(
            WSUtil.WS_ENCRYPTION_PARTS,
            "{Element}{" +
            WSU_NS +
            "}Timestamp;" +
            "{Content}{http://schemas.xmlsoap.org/soap/envelope/}Body",
        );


        requestCfg.put(WSUtil.WS_SIGNATURE_USER, alisForSignature);
        requestCfg.put(WSUtil.WS_PASSWORD_TYPE, WSUtil.WS_PW_TEXT);

        requestCfg.put(WSUtil.WS_SIG_DIGEST_ALGO,
"http://www.w3.org/2001/04/xmlenc#sha256");


        // define signature properties
        requestCfg.put(WSUtil.WS_SIG_PROP_KEYSTORE_TYPE, "managed");
        requestCfg.put(WSUtil.WS_SIG_PROP_KEYSTORE_ALIAS, alisForSignature);

        requestCfg.put(WSUtil.WS_SIGNATURE_PARTS,
"{Element}{http://schemas.xmlsoap.org/soap/envelope/}Body");

```

```

    requestCfg.put(WSUtil.WS_SIG_KEY_ID,
WSUtil.KEY_ID_TYPE_DIRECT_REFERENCE);

    requestCfg.put(WSUtil.WS_SECRETS_MAP, secretsMap);

    //response-config
    var responseCfg = new HashMap();

    responseCfg.put(WSUtil.WS_ACTION, WSUtil.WS_TIMESTAMP);

    WSUtil.setWSSecurityConfig(svc.serviceClient, requestCfg, responseCfg);
    // Setting WS security

    return svc.serviceClient.runTransaction(parameter.request);
},

```

Step 3: Please refer to below screenshots and make changes in libCybersource.js

```

278 -
279 483     setEndpoint: function (service) {
280 484         var endpoint = CybersourceHelper.getEndpoint();
281 485         var Logger = dw.system.Logger.getLogger('Cybersource');
282 486         Logger.debug('Connection to system "{}"', endpoint);
283 -         var Stub = require('dw/rpc/Stub');
284 -
287 +         var Port = require('dw/ws/Port');
288 +         var WSUtil = require('dw/ws/WSUtil');
285 489         switch (endpoint) {
286 490             case 'Production':
287 -                 service._setProperty(Stub.ENDPOINT_ADDRESS_PROPERTY, 'https://ics2wsa.ic3.com/commerce/1.x/transactionProcessor');
288 491 +                 WSUtil.setProperty(Port.ENDPOINT_ADDRESS_PROPERTY, 'https://ics2wsa.ic3.com/commerce/1.x/transactionProcessor', service);
288 492                 break;
289 493             case 'Test':
290 -                 service._setProperty(Stub.ENDPOINT_ADDRESS_PROPERTY, 'https://ics2wstesta.ic3.com/commerce/1.x/transactionProcessor');
291 494 +                 WSUtil.setProperty(Port.ENDPOINT_ADDRESS_PROPERTY, 'https://ics2wstesta.ic3.com/commerce/1.x/transactionProcessor', service);
291 495                 break;
292 496             default:
293 497 +                 // eslint-disable-next-line
293 498                 throw 'Undefined Cybersource Endpoint "' + endpoint + '"';
294 499         }
295 500     },
296 501

```

```
296 -         var Stub = require('dw/rpc/Stub');
296 +         var Port = require('dw/ws/Port');
297 +         var WSUtil = require('dw/ws/WSUtil');
297 298
298 299         switch ( endpoint ) {
299 300             case "Production":
300 -                 service._setProperty(Stub.ENDPOINT_ADDRESS_PROPERTY, 'https://ics2wsa.ic3.com/commerce/1.x/transactionProcessor');
301 +                 WSUtil.setProperty(Port.ENDPOINT_ADDRESS_PROPERTY, 'https://ics2wsa.ic3.com/commerce/1.x/transactionProcessor');
301 -                 service);
301 302             break;
302 303             case "Test" :
303 -                 service._setProperty(Stub.ENDPOINT_ADDRESS_PROPERTY, 'https://ics2wstesta.ic3.com/commerce/1.x/transactionProcessor');
304 +                 WSUtil.setProperty(Port.ENDPOINT_ADDRESS_PROPERTY, 'https://ics2wstesta.ic3.com/commerce/1.x/transactionProcessor');
304 -                 service);
304 305             break;
305 306             default:
306 307                 throw "Undefined Cybersource Endpoint \"" + endpoint + "\"";
307 308         }
307 309     }
308 310 }
```

Step 4: Post completing the above changes please make the changes by referring to [section 1](#).

NOTE: webreference has been updated to webreferences2 in later versions of our cartridge. So, changes added to replace webreferences2 in section 1 to be considered as webreferences in older versions.